

A PROJECT REPORT ON

**Classification of Segments to Be Translated or
Transliterated**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aditya Shah	Exam No: B400050314
Atharva Date	Exam No: B400050367
Maheshwar Bhosale	Exam No: B400050342
Samarth Mali	Exam No: B400050488



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
Classification of Segments to Be Translated or Transliterated

Submitted by

Aditya Shah	Exam No: B400050314
Atharva Date	Exam No: B400050367
Maheshwar Bhosale	Exam No: B400050342
Samarth Mali	Exam No: B400050488

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. G. V. Kale** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. G. V. Kale
Internal Guide
Dept. of Computer Engg.

Dr. G.V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Classification of Segments to Be Translated or Transliterated**’.*

*We would like to take this opportunity to thank **Dr G.V. Kale** for giving us all the help and guidance we needed. We are really grateful to her for the kind support. Her valuable suggestions and mentorship were very helpful.*

*We are also grateful to **Dr G.V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

Aditya Shah
Atharva Date
Maheshwar Bhosale
Samarth Mali
(B.E. Computer Engg.)

ABSTRACT

In multilingual natural language processing, accurately deciding whether a text segment should be translated or transliterated is essential for preserving meaning and maintaining consistency. This project presents a two-layered system designed to classify each token based on this distinction. The first layer is a RoBERTa-based Named Entity Recognition (NER) model, fine-tuned to identify tokens—such as names, places, and specific terms—that typically require transliteration. To improve recall and handle edge cases, we introduce a second layer using Part-of-Speech (POS) tagging through the NLTK library. This layer acts as a safety net, capturing tokens that may be overlooked by the NER model.

To support model training, we generated a high-quality synthetic dataset using an agentic chain of large language models. This approach allowed us to simulate a wide range of realistic, multilingual cases with controlled annotation, bypassing the limitations of scarce labeled data. The resulting system is lightweight, efficient, and reliable, offering improved accuracy in segment-level classification. It can be integrated into existing translation pipelines where the need to differentiate between translation and transliteration is critical.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	3
1.5.1	Hybrid Two-Layer Classification Architecture	3
1.5.2	Synthetic Data Generation Pipeline	3
2	Literature Survey	5
2.1	Translation and Transliteration	6
2.2	Token Classification for Translation and Transliteration	7
2.2.1	Named Entity Recognition (NER) for Disambiguation	7
2.2.2	Part-of-Speech (POS) Tagging for Transliteration Disambiguation	7
2.2.3	Transformer-Based NER for Token Classification	8
2.2.4	Unsupervised and Hybrid Approaches	8
2.2.5	Neural Network-Based Techniques	8
2.2.6	Multilingual Resources and Datasets	8
2.3	Synthetic Dataset Generation	8
3	Software Requirements Specification	11
3.1	Assumptions and Dependencies	12
3.1.1	Assumptions	12
3.1.2	Dependencies	12
3.2	Functional Requirements	13
3.2.1	System Feature 1: Named Entity Recognition (NER)	13
3.2.2	System Feature 2: Part-of-Speech (POS) Tagging	13
3.2.3	System Feature 3: Synthetic Data Generation	13
3.3	External Interface Requirements	14
3.3.1	User Interfaces	14
3.3.2	Hardware Interfaces	14
3.3.3	Software Interfaces	14
3.3.4	Communication Interfaces	14
3.4	System Requirements	15
3.4.1	Database Requirements	15
3.4.2	Software Requirements (Platform Choice)	15
3.4.3	Hardware Requirements	15
3.5	Analysis Models: SDLC Model to be applied	16

4	System Design	17
4.1	System Architecture	18
4.2	Mathematical Model	18
4.3	Data Flow Diagrams	19
4.4	Entity Relationship Diagram	22
4.5	UML Diagrams	22
5	Project Plan	23
5.1	Project Estimate	24
5.1.1	Reconciled Estimates	24
5.1.2	Project Resources	24
5.2	Risk Management	25
5.2.1	Risk Identification	25
5.2.2	Risk Analysis	25
5.2.3	Overview of Risk Mitigation, Monitoring, Management	25
5.3	Project Schedule	27
5.3.1	Project Task Set	27
5.3.2	Task Network	27
5.3.3	Timeline Chart	27
5.4	Team Organization	28
5.4.1	Team structure	28
5.4.2	Management reporting and communication	28
6	Project Implementation	29
6.1	Overview of Project Modules	30
6.2	Tools and Technologies Used	30
6.3	Algorithm Details	31
6.3.1	Algorithm 1: Synthetic Data Generation with Multi-Agent LLM Prompts	31
6.3.2	Algorithm 2: Named Entity Recognition (NER) Model Training	31
6.3.3	Algorithm 3: POS-Aware Post-Validation	33
6.3.4	Algorithm 4: Inference Pipeline	33
7	Software Testing	35
7.1	Type of Testing	36
7.2	Test cases & Test Results	36
8	Results	38
8.1	Outcomes	39
8.1.1	Synthetic Data Diversity	39
8.1.2	NER Model Evaluation	39
8.1.3	POS Tagging Models	39
8.1.4	Conclusion of Results	39
8.2	Screen Shots	40
9	Conclusions	41
9.1	Conclusions	42
9.2	Future Work	42
9.3	Applications	42

10 Appendix	43
10.1 Appendix A	44
10.2 Appendix B	45
11 References	46

List of Figures

4.1	Synthetic Data Generation	19
4.2	NER Flow	20
4.3	Use Case	20
4.4	Research Area	21
4.5	Activity Diagram	22
8.1	Super-LLM	40
8.2	UI	40
10.1	Screenshot	45

CHAPTER 1

INTRODUCTION

1.1 Overview

In an increasingly multilingual digital world, the handling of language-specific nuances like translation and transliteration has become essential for natural language processing (NLP) systems. This project, carried out as part of a research internship at Reverie Language Technologies, focuses on building a system that can automatically classify whether a given text segment should be translated or transliterated. By combining machine learning techniques such as Named Entity Recognition (NER) and Part-of-Speech (POS) tagging, the system addresses a critical bottleneck in multilingual processing pipelines, improving the quality and precision of downstream language tasks. The work specifically integrates models such as RoBERTa for NER, and the NLTK toolkit for POS, within a two-layered hybrid architecture to achieve fine-grained classification of text segments

1.2 Motivation

The motivation behind this project arises from the real-world need to preserve both the meaning and cultural authenticity of language content across diverse languages and scripts. Simple translation often fails when dealing with named entities like proper names, product brands, or culturally specific terms that are better handled through transliteration. Misclassifying these segments can result in loss of meaning, readability issues, or even user distrust. Furthermore, existing systems often struggle due to the lack of high-quality labeled datasets, particularly for specialized or low-resource domains. Motivated by these challenges, our team aimed to develop a robust classification system that could overcome data limitations through innovative synthetic data generation and deliver a reliable solution for multilingual applications

1.3 Problem Definition and Objectives

The core problem addressed by this project is automatically identifying which text segments in a multilingual document should be translated and which should be transliterated. Given a continuous stream of text, the system must segment it intelligently and apply the correct language transformation approach to each part

Objectives of the project are :

1. Designing a hybrid model architecture combining NER and POS tagging to distinguish transliteration candidates.
2. Selecting optimal machine learning models (RoBERTa for NER, NLTK for POS) to balance accuracy and efficiency.
3. Overcoming the lack of domain-specific training data by developing a synthetic data generation pipeline using advanced Large Language Model (LLM) - based techniques.
4. Validating the system's performance through extensive experimentation to ensure robustness across varied language data..

1.4 Project Scope & Limitations

This project focuses on segment-level classification for translation and transliteration in multilingual text, particularly targeting domains relevant to Reverie’s language technologies. The system is designed to handle Indian languages and English, reflecting the practical needs of regional language applications. However, the project operates under several limitations:

1. The classification system relies heavily on the quality and diversity of synthetic data generated, which may introduce bias or gaps if not carefully tuned.
2. While the selected RoBERTa and NLTK tools provide a strong balance between accuracy and computational efficiency, they might not capture all domain-specific subtleties, especially for rare or emerging entities.
3. The system currently focuses only on binary classification (translate vs. transliterate) and does not handle more nuanced transformations like localization or cultural adaptation.
4. Resource constraints limited the experimentation with larger-scale or multilingual LLMs, focusing the implementation on practical, deployable solutions rather than theoretical explorations.

1.5 Methodologies of Problem solving

1.5.1 Hybrid Two-Layer Classification Architecture

The core system design employs a two-layer classification architecture.

- **First Layer: Named Entity Recognition (NER)**

We selected the RoBERTa transformer-based model due to its superior contextual understanding and performance in named entity detection tasks. RoBERTa identifies potential segments in the input text that are likely to be named entities, such as personal names, places, and organizations — all strong candidates for transliteration rather than translation.

- **Second Layer: Part-of-Speech (POS) Tagging**

The outputs from the first layer are further validated using the NLTK POS tagger. POS tagging provides a linguistic check to ensure that non-entity tokens mistakenly marked in the first layer (such as common nouns or adjectives) are filtered out or corrected. This dual-layer approach combines the statistical power of transformer models with lightweight linguistic validation for higher precision.

1.5.2 Synthetic Data Generation Pipeline

One of the most significant challenges was the scarcity of high-quality, domain-specific labeled data. To overcome this, we developed a custom synthetic data generation pipeline based on Large Language Models (LLMs):

- We implemented an Agentic-AI approach that coordinates multiple generative agents, each tasked with producing specific linguistic patterns, domain terms, or rare entity combinations.
- We implemented adaptive model swapping to alternate between LLMs like GPT-3.5, LLaMA, or smaller finetuned models, ensuring diversity in text generation while avoiding overfitting to a particular model's biases.
- To ensure variety and avoid redundancy, we integrated hyperparameter tuning strategies (e.g., temperature scaling, top-k sampling, prompt variation) during generation, maximizing lexical, syntactic, and semantic diversity in the synthetic dataset.

This synthetic dataset was used to pre-train and finetune our models, boosting their performance even in domain-constrained environments.

CHAPTER 2

LITERATURE SURVEY

2.1 Translation and Transliteration

In multilingual neural machine translation (NMT), one of the most significant challenges is distinguishing between content that should be translated (i.e., semantic conversion) and content that must be transliterated (i.e., phonetic preservation). This distinction becomes crucial when dealing with named entities, such as person names, locations, or domain-specific terms, which require transliteration to preserve their identity, as opposed to common nouns or verbs, which should be translated for meaningful communication in the target language. A failure to correctly classify these segments can lead to semantic ambiguity and degradation of translation quality, particularly in languages with rich morphology, like Hindi.

For example, the sentence "Mr. Snow Bark went to office" contains Bark as a surname, which should be transliterated to preserve its identity, whereas "Dogs bark in panic situations" requires translation of bark as a verb. While models like Google Translate have demonstrated competence in resolving such ambiguities, many open-source and resource-constrained systems struggle with accurate translation-transliteration decision-making. This issue has prompted extensive research into developing models that can automatically identify which segments of a sentence should be translated and which should be transliterated.

The treatment of named entities (NE) as a distinct subproblem within NMT has been a significant area of research. Grundkiewicz and Heafield [21] proposed a sequence-to-sequence approach for named entity transliteration using an attention-based encoder-decoder architecture. Their work, presented in the context of the NEWS 2018 shared task, demonstrated that models trained specifically for transliteration outperformed generic translation models when handling named entities, especially for rare or out-of-vocabulary terms. This approach highlights the importance of dedicating separate processing pathways for transliteration tasks.

Building upon this, Sharma et al. [13] introduced enhancements to traditional NMT pipelines by incorporating a preprocessing step to isolate named entities. Their method involved constructing entity dictionaries and utilizing heuristic rules to identify proper nouns before translation, ensuring that phonetic preservation could be handled separately from the semantic translation. Their experiments showed that this modular approach improved translation quality by achieving over 99% accuracy in processing person, location, and organization names across Hindi-English datasets.

Context-sensitive models have proven to be highly effective in disambiguating terms that could be either translated or transliterated. Hu et al. [14] proposed DEEP (Denoising Entity Pretraining), a novel pretraining method that integrates large-scale monolingual corpora and external knowledge bases to enhance NMT. By pretraining models on vast amounts of unannotated data, DEEP allows the system to align entity mentions with external knowledge, improving its ability to handle rare or out-of-vocabulary terms. Crucially, this pretraining framework enables the model to classify terms for translation or transliteration based on both their semantic type and contextual relevance.

Similarly, Ohtani et al. [15] introduced a graph-based encoder that incorporates coreference resolution into NMT. By modeling long-range dependencies and maintaining referential consistency, their approach significantly improved the system's ability to resolve homographs—terms such as bark that have different meanings depending on the context. Their work suggests that higher-level contextual understanding, such as discourse-level coherence, plays a key role in making accurate translation-transliteration

decisions.

A related line of research focuses on transliteration in Romanized or code-mixed texts, where phonetic ambiguity is common. Jain et al. [16] proposed a context-aware transliteration framework for Romanized South Asian languages, where words like *ki* require contextual disambiguation. Their model leveraged full-sentence context to resolve these ambiguities, suggesting that contextual embeddings can significantly improve transliteration quality. While their research was primarily concerned with transliteration, their approach has direct relevance to NMT systems tasked with translation-transliteration disambiguation.

2.2 Token Classification for Translation and Transliteration

In multilingual neural machine translation (NMT), accurately distinguishing between tokens that require semantic translation and those necessitating phonetic transliteration is crucial. Misclassification can lead to semantic inaccuracies, especially in morphologically rich languages like Hindi. This subsection explores methodologies developed to address this challenge, emphasizing token-level disambiguation strategies.

2.2.1 Named Entity Recognition (NER) for Disambiguation

Early approaches leveraged Named Entity Recognition (NER) to identify tokens that should be transliterated. Babych and Hartley (2004) demonstrated that integrating automatic NER into machine translation systems improved the handling of named entities by distinguishing them from common nouns, thereby enhancing translation accuracy [17]. Similarly, Wentland et al. (2008) introduced HeiNER, a multilingual lexical resource containing disambiguated English named entities with translations and transliterations across 15 languages. HeiNER utilized cross-lingual information from Wikipedia to construct a reliable database for NER, achieving a high precision of 0.95 [18].

2.2.2 Part-of-Speech (POS) Tagging for Transliteration Disambiguation

POS tagging has also been used as a lightweight but effective method to separate tokens likely to be named entities (e.g., proper nouns) from those requiring semantic translation (e.g., verbs, adjectives). Yin et al. (2019) proposed a tag-enhanced neural machine translation system that jointly models translation and POS tagging using multi-task learning. Their model demonstrated improved performance by using syntactic clues from POS tags to resolve translation ambiguities, particularly for unknown or ambiguous tokens [24].

Hlaing et al. (2022) further incorporated POS/UPOS tags as part of a prompt-based neural machine translation (AP-NMT) framework. Their method used these tags to guide translation, especially in low-resource scenarios, achieving gains in BLEU scores over baselines for several language pairs [25].

2.2.3 Transformer-Based NER for Token Classification

The emergence of transformer-based models like BERT has opened new avenues for token-level classification tasks. Fine-tuning BERT for NER allows the model to leverage deep contextual embeddings, significantly improving entity recognition accuracy. Devlin et al. (2019) showed that BERT-based models outperform prior NER systems on multiple benchmarks by capturing context across sentence boundaries [26].

Such models have been adapted in NMT pipelines to identify named entities that should be transliterated instead of translated, especially in noisy or code-mixed inputs. They are particularly effective in handling ambiguous terms such as “bark” or “ki,” where both syntactic position and surrounding context are critical for correct disambiguation.

2.2.4 Unsupervised and Hybrid Approaches

Chen and Skiena (2016) proposed an unsupervised transliteration model capable of generating transliterations for arbitrary strings across 69 languages. Their model effectively detected “false friends”—words that appear similar across languages but differ in meaning—by analyzing pronunciation similarities, thereby aiding in distinguishing between translation and transliteration candidates [19]. Balabantaray et al. (2009) presented a hybrid approach combining statistical methods with linguistic knowledge to transliterate named entities. Their method focused on the linguistic aspects of conjuncts and diphthongs in source and target languages, enhancing the transliteration process [20].

2.2.5 Neural Network-Based Techniques

Grundkiewicz and Heafield (2018) treated named entity transliteration as a sequence-to-sequence task using attentional encoder-decoder architectures. Their approach, applied in the NEWS 2018 shared task, achieved competitive results across language pairs, illustrating the efficacy of neural models trained specifically for transliteration tasks [21]. Merhav and Ash (2018) analyzed design challenges in developing multilingual named entity transliteration systems, comparing traditional weighted finite state transducer (WFST) approaches with neural methods like encoder-decoder RNNs and Transformers. Their empirical evaluations highlighted the advantages and limitations of each method, contributing to the understanding of transliteration in NMT [22].

2.2.6 Multilingual Resources and Datasets

Sälevä and Lignos (2022) introduced ParaNames, a massively multilingual entity name corpus encompassing 118 million names across 400 languages. ParaNames provides standardized entity types and is instrumental in training models for name translation and transliteration, serving as a valuable resource for multilingual language processing tasks [23].

2.3 Synthetic Dataset Generation

In natural language processing (NLP), the availability of large, high-quality annotated datasets is often a limiting factor for developing robust models. To address this challenge,

synthetic data generation has emerged as a practical solution for augmenting existing corpora or creating new datasets entirely. This section explores various approaches employed to generate synthetic textual data, focusing on methods that produce coherent, diverse, and task-relevant sentences to support downstream NLP tasks.

Back-translation is a widely used technique in NLP for augmenting datasets by translating sentences from the source language to a target language, and then translating them back to the source. This results in slight modifications of the original sentence, introducing variability while preserving its semantic meaning. Originally introduced by Sennrich et al. [1] to improve neural machine translation models, this approach has since been applied to a variety of NLP tasks to increase data diversity and enhance model generalization.

Another simple rule-based augmentation method is synonym substitution, where words in a sentence are replaced with their synonyms using lexical resources such as WordNet or word embeddings. This technique introduces lexical variation without altering the core semantics of the sentence, thereby enriching the training data. Wei and Zou [2] included this approach as part of their EDA (Easy Data Augmentation) framework, showing that even minimal lexical transformations can improve text classification performance when labeled data is limited.

Recent advances in large language models have enabled the generation of high-quality synthetic text data using prompt-based strategies. In zero-shot prompting, the model generates data without any task-specific examples, relying solely on its pre-trained knowledge. One-shot and few-shot prompting further enhance this by providing one or a few exemplars to guide generation. These techniques were first systematically demonstrated in the context of GPT-3 by Brown et al. [3], showcasing the ability of large-scale models to generalize and synthesize task-relevant text from minimal input. Further work by Puri and Catanzaro [4] explored prompt-based generation specifically for synthetic text classification datasets, confirming the practical utility of LLMs in data-scarce NLP settings.

Role-based prompting is another promising approach for generating synthetic data, where the model is instructed to assume a specific persona or role, such as "You are a renowned poet" or "You are a highly experienced doctor." This method effectively steers the model's responses to align with the expected expertise or creativity, thereby producing contextually relevant and specialized outputs. This technique has shown significant potential, as demonstrated by Kong et al. [5], who explored its effectiveness in enhancing reasoning across diverse tasks.

Building upon role-based prompting, AI agents have been increasingly utilized for generating synthetic text data. These agents, often powered by large language models, can autonomously generate text by interacting with users or other agents in a specific context. Such agents have been effectively employed for generating varied and contextually rich datasets. For instance, [6] introduced a multi-agent framework designed to generate a dataset of questions that mimic customer queries and reverse-engineer alternate questions from the responses for trajectory verification. Similarly, [7] proposed a methodology for generating synthetic Preference Optimization datasets using multi-agent workflows, automating and enhancing the dataset generation process. Meanwhile, [8] demonstrated how multi-agent workflows can be employed to generate synthetic datasets for training conversational agents, showcasing the versatility of AI agents in diverse applications. These works illustrate the significant potential of AI agents for generating high-quality, specialized data, offering a promising avenue for enhancing model performance across

various NLP tasks.

Another technique explored for synthetic text generation is text seeding, wherein a generation process is initiated using keyphrases or text fragments to guide large language models in producing coherent and contextually appropriate outputs. This method has been notably employed by Nagesh et al. [9] through their Differentially Private Keyphrase Prompt Seeding (DP-KPS) framework. The approach leverages differentially private mechanisms to extract representative keyphrases from sensitive data, which are then used to seed language model prompts, enabling the generation of synthetic text that preserves both semantic characteristics and privacy constraints. Their work demonstrates that such seeded prompts can yield synthetic datasets effective for downstream tasks, especially in domains like healthcare where data sharing is highly restricted.

An additional strategy for enhancing diversity in synthetic text generation involves dynamically adjusting the temperature parameter during the decoding phase of large language models (LLMs). The temperature controls the randomness in token sampling—lower values yield more deterministic outputs, while higher values introduce greater variability. Although traditionally a fixed temperature setting is used, recent research has explored varying this parameter to achieve a better trade-off between creativity and coherence. Zhang et al. [10] proposed an Entropy-based Dynamic Temperature (EDT) sampling method, where the temperature is modulated based on the entropy of the token distribution. This enables the model to produce diverse outputs in uncertain contexts and more focused responses when confident, leading to superior performance across multiple generation tasks. Complementarily, Renze and Guven [11] empirically evaluated the role of temperature variation in problem-solving tasks and found that while it affects output diversity, it has limited impact on accuracy. These findings establish dynamic temperature modulation as an effective tool for generating varied yet high-quality synthetic text datasets.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

This section outlines the assumptions and dependencies of the project, along with the user classes and characteristics.

3.1.1 Assumptions

- **Pre-trained Models:** The project assumes access to pre-trained transformer models, particularly RoBERTa from **Huggingface Transformers**, for fine-tuning on the transliteration and translation tasks.
- **Cloud Infrastructure:** The system assumes access to cloud services like **Azure** for hosting and **Cloud GPUs** for training and inference.
- **Sufficient Data:** The effectiveness of the model depends on the availability of a synthetic dataset, generated using an agentic chain of LLMs, which is assumed to approximate real-world language usage.
- **Internet Connectivity:** The project assumes stable internet access for integrating external APIs, such as **OpenAI**, **Claude**, and **Gemini**, to generate data and fine-tune models.

3.1.2 Dependencies

- **Huggingface Transformers:** The project relies on **Huggingface Transformers** for the RoBERTa model used in Named Entity Recognition (NER).
- **PyTorch and CUDA:** The system uses **PyTorch** for model training, with **CUDA** and **CuBLAS** for GPU acceleration.
- **NLTK:** NLTK is used for the Part-of-Speech (POS) tagging layer, aiding in token classification.
- **Cloud Services:** The project depends on **Azure** for storage, deployment, and scaling, along with **Cloudinary** for image storage and **Docker** for containerization.
- **APIs and Data Generation:** The system integrates APIs from **OpenAI**, **Claude**, and **Gemini** to generate the synthetic dataset and fine-tune models.
- **Docker:** **Docker** is used for containerizing the application, ensuring compatibility across different environments.

3.2 Functional Requirements

3.2.1 System Feature 1: Named Entity Recognition (NER)

The core functionality of the system relies on the Named Entity Recognition (NER) layer, which performs the essential task of classifying tokens. The NER model, trained on **RoBERTa**, identifies entities such as names, places, organizations, and other critical elements that need to be either translated or transliterated. The model is fine-tuned to detect entities in a wide range of languages and syntactic structures, ensuring accurate classification.

- The NER layer is responsible for identifying named entities such as locations, organizations, and names.
- It uses a fine-tuned **RoBERTa** model to identify contextually relevant entities.
- This layer provides the backbone for accurate classification and decision-making regarding translation or transliteration.

3.2.2 System Feature 2: Part-of-Speech (POS) Tagging

The Part-of-Speech (POS) tagging layer serves as a safety net for the system. While the NER layer handles most of the entity identification, POS tagging provides a backup to ensure any missed tokens are caught. By using **NLTK**, the POS layer labels the tokens with grammatical information such as nouns, verbs, adjectives, and adverbs. This layer improves the system's robustness and accuracy in edge cases where the NER layer might fail.

- The POS layer labels tokens with grammatical categories, ensuring all tokens are classified correctly.
- Using **NLTK**, the POS layer supports the NER layer by catching missed tokens.
- It improves the overall system accuracy and robustness.

3.2.3 System Feature 3: Synthetic Data Generation

The system utilizes **agentic chains of LLMs** for generating synthetic data, which helps augment the training dataset and improve model performance. This data is diverse, covering various languages, sentence structures, and contexts, making the model more adaptable to real-world scenarios.

- The synthetic data is generated using LLMs to cover a wide range of languages and complex sentence structures.
- It enhances model robustness by introducing diversity in tokenization, syntax, and named entities, aiding in accurate translation or transliteration.
- Continuous generation and adaptation of synthetic data are crucial to avoid bias and improve performance across diverse use cases.

3.3 External Interface Requirements

3.3.1 User Interfaces

The project features a full-stack web application with a React-based frontend and a Django backend. Users interact through a clean, responsive interface where they can input text segments for classification. The frontend communicates with cloud-hosted model APIs to receive classification results — whether a given segment should be translated or transliterated. The UI is designed for clarity, ease of use, and responsiveness, catering to researchers, linguists, and developers. The model can also be used directly using terminal cURL commands.

3.3.2 Hardware Interfaces

The system is optimized for machines with GPU acceleration. Training and large-scale inference benefit from NVIDIA GPUs with CUDA/cuBLAS support. Recommended specifications include at least 16 GB RAM, a modern multi-core CPU, and a GPU with a minimum of 8 GB VRAM. However, for API consumers, only a standard client device with internet access is sufficient.

3.3.3 Software Interfaces

The solution integrates the following software components:

- **Operating System:** Compatible with Linux, macOS, or Windows.
- **Programming Language:** Python 3.8+ for backend/model; JavaScript (React) for frontend
- **Core Libraries:** Huggingface Transformers, PyTorch, NLTK, Pandas, Polars, Seaborn
- **LLM Tooling and Orchestration:** LangChain, OpenAI, Claude, Gemini, Groq, Mistral, LlamaIndex, MetaLLaMA
- **Deployment Tools:** Docker, Flask (for prototyping), Django (for production APIs)
- **Frontend Framework:** ReactJS

All tools are either open-source or available via secure API access. Deployment assumes cloud-based hosting with public API access secured via authentication.

3.3.4 Communication Interfaces

The frontend and backend communicate via REST APIs. Backend APIs also connect with LLM services and model inference endpoints hosted in the cloud. All communication is performed over HTTPS to ensure secure data transmission. The modular API architecture allows swapping in alternative models or services without major refactoring.

3.4 System Requirements

3.4.1 Database Requirements

The application does not rely on a heavy database, as its core functionality is focused on real-time classification of textual segments. However, a lightweight SQL-based database like SQLite or PostgreSQL is used for storing user-related data, API usage logs, and system analytics. This ensures efficient session management and allows monitoring system usage patterns.

3.4.2 Software Requirements (Platform Choice)

The project leverages a modern software stack combining full-stack web development and machine learning infrastructure:

- **Frontend:** Built with React.js, providing a responsive and intuitive UI for user interaction.
- **Backend:** Developed using Django REST Framework to handle API routing, user sessions, and model interaction.
- **Model APIs:** Flask-based APIs serve the model hosted on a cloud platform with GPU support.
- **Libraries and Frameworks:** Huggingface Transformers, PyTorch, NLTK, LangChain, LlamaIndex, Pandas, Polars, Seaborn, and more.
- **Development Tools:** Jupyter Notebook for experimentation, Docker for containerized deployment.
- **Programming Language:** Python

The stack is platform-independent and supports Linux, Windows, and macOS environments.

3.4.3 Hardware Requirements

To ensure smooth performance, especially during training and inference with the RoBERTa model and LLM-based synthetic data generation, access to GPUs is crucial. Recommended cloud infrastructure includes:

- A CUDA-compatible GPU (e.g., NVIDIA T4, A100, or similar)
- At least 16 GB of RAM
- Minimum of 50 GB of available storage

Local development and testing of the POS layer and interface components can be carried out on a standard CPU machine.

3.5 Analysis Models: SDLC Model to be applied

For the development of this project, we adopted the **Incremental Software Development Life Cycle (SDLC)** model. This model was well-suited to our task, allowing us to build, evaluate, and integrate each core functionality step-by-step.

Initially, we focused on building the core classification model using a Named Entity Recognition (NER) approach, training RoBERTa on synthetically generated data. Once this core segment classifier was functional, we incrementally layered the secondary Part-of-Speech (POS) based safety net using NLTK, ensuring that even overlooked tokens were handled effectively.

In the next phase, we developed a web-based interface using React for the frontend and Django for the backend. Integration of the trained model APIs (hosted via Flask on a cloud GPU instance) followed, allowing real-time interaction with user inputs through the website. The incremental model allowed parallel progress in model fine-tuning, synthetic data enhancement (via an agentic chain of LLMs), and API/UX development.

This modular, iterative approach provided agility in experimentation, continuous testing, and clear tracking of progress while maintaining alignment with the evolving requirements of transliteration-focused token classification.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The proposed system is architected to identify and label segments within an English sentence that should undergo transliteration instead of translation, particularly focusing on named entities, proper nouns, and culture-specific terminology. The system is composed of four primary modules: *Synthetic Data Generation*, *Model Training*, *POS-based Validation*, and *Inference Pipeline*.

1. **Synthetic Data Generation Module:** This module utilizes a multi-agent, LLM-backed framework to generate labeled synthetic data simulating code-mixed Indic language settings. To ensure diversity, the generation process includes:
 - Randomized sampling of temperature and beam search width to vary lexical structure.
 - Few-shot prompt engineering with k -combinations of manually curated examples.
 - Agent role conditioning and auxiliary keyword insertion to diversify semantics and syntax.
2. **NER Model Training Module:** Transformer-based models (Google MURIL, BERT, DistilBERT, RoBERTa) are fine-tuned on the synthetic dataset $\mathcal{D}_{syn} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$, where $\mathbf{x}^{(i)}$ is a tokenized sentence and $\mathbf{y}^{(i)}$ is a sequence of token-wise binary labels indicating transliteration requirements.
3. **POS-based Validation Module:** To reinforce reliability, a secondary POS-based classifier is applied. Let $\mathcal{T}^{(i)} = \{t_1^{(i)}, \dots, t_n^{(i)}\}$ be the POS tags for sentence $\mathbf{x}^{(i)}$. Tokens with $t_j^{(i)} \in \{\text{PROPN}, \text{NOUN}\}$ override the NER decision if initially labeled for translation.
4. **Inference Pipeline:** At inference time, an input sentence $\mathbf{x} = \{w_1, \dots, w_n\}$ is passed sequentially through the NER classifier and POS validator to obtain $\hat{\mathbf{y}} = \{\hat{y}_1, \dots, \hat{y}_n\}$.

4.2 Mathematical Model

Let an input sentence be defined as a sequence of n tokens:

$$\mathbf{x} = \{w_1, w_2, \dots, w_n\}, \quad w_i \in \Sigma^*$$

Each token w_i is to be classified into a binary label $y_i \in \{0, 1\}$, where:

$$y_i = \{1 \text{ if } w_i \text{ should be transliterated}, 0 \text{ if } w_i \text{ should be translated}.$$

We define:

- $\mathcal{M}_{NER} : \mathbf{x} \mapsto \hat{\mathbf{y}}^{NER}$, a fine-tuned NER model mapping tokens to binary labels.
- $\mathcal{M}_{POS} : \mathbf{x} \mapsto \mathbf{t}$, a POS tagging model where $\mathbf{t} = \{t_1, \dots, t_n\}$ and $t_i \in \mathcal{T}$ (set of POS tags).

To improve classification robustness, we define the final classification rule:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \hat{y}_i^{NER} = 1 \\ 0 & \text{if } \hat{y}_i^{NER} = 0 \text{ and } t_i \in \{\text{PROPN}, \text{NOUN}\} \\ 0 & \text{otherwise} \end{cases}$$

The learning objective is defined via the binary cross-entropy loss:

$$\mathcal{L}(\theta) = - \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(f_{\theta}(w_i))$ is the model's predicted probability (after applying softmax or sigmoid) for transliteration, and θ denotes the model parameters.

4.3 Data Flow Diagrams

A description of each major software function, along with data flow (structured analysis) or class hierarchy (Analysis Class diagram with class description for object oriented system) is presented.

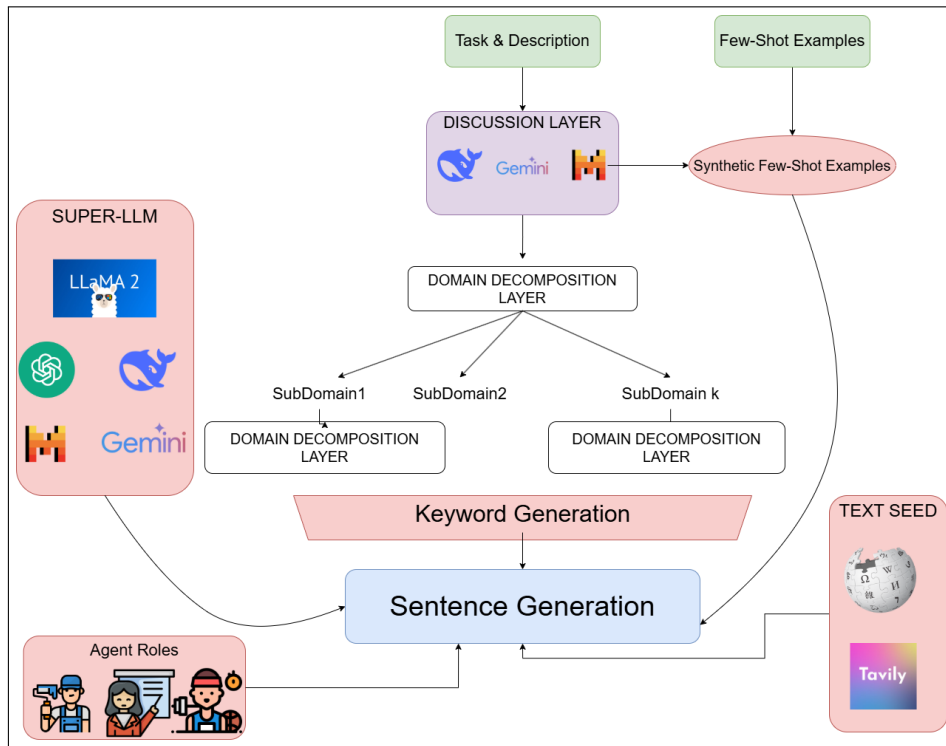


Figure 4.1: Synthetic Data Generation

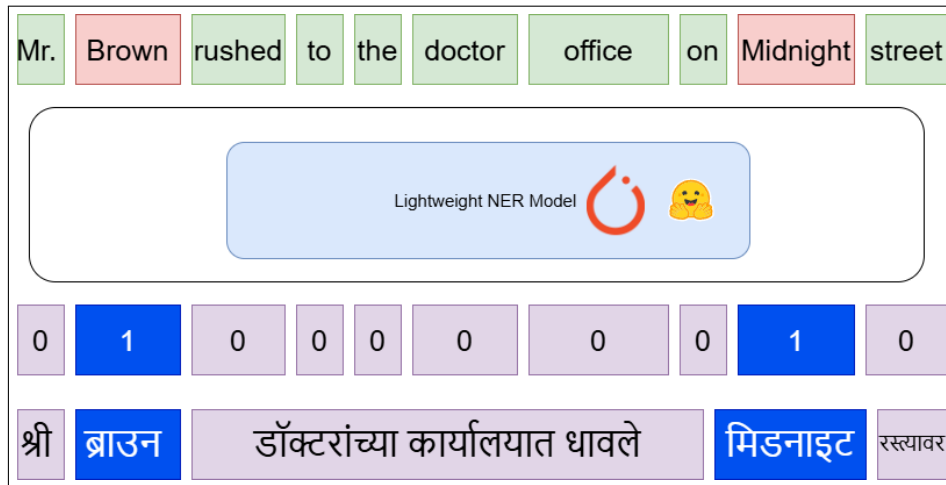


Figure 4.2: NER Flow

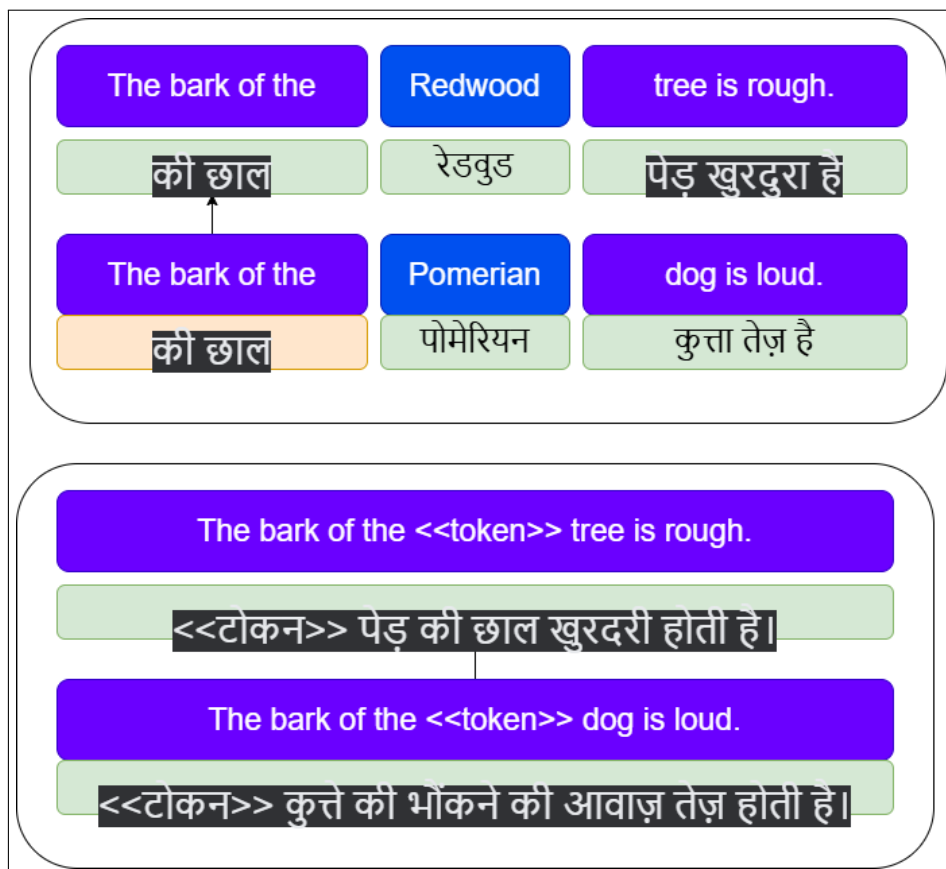


Figure 4.3: Use Case

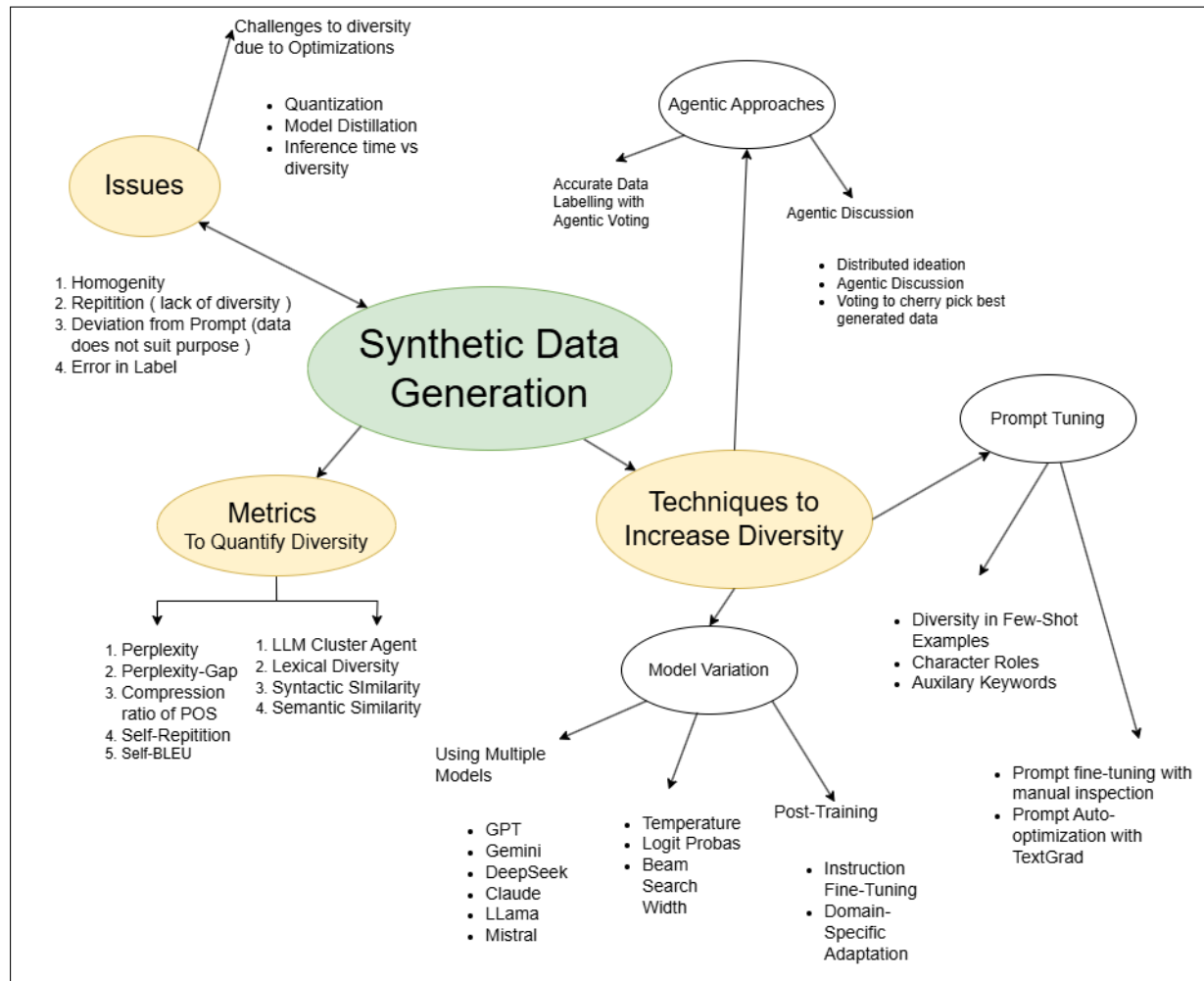


Figure 4.4: Research Area

4.4 Entity Relationship Diagram

Here, we have shown how two different entities will interact using the system architecture.

4.5 UML Diagrams

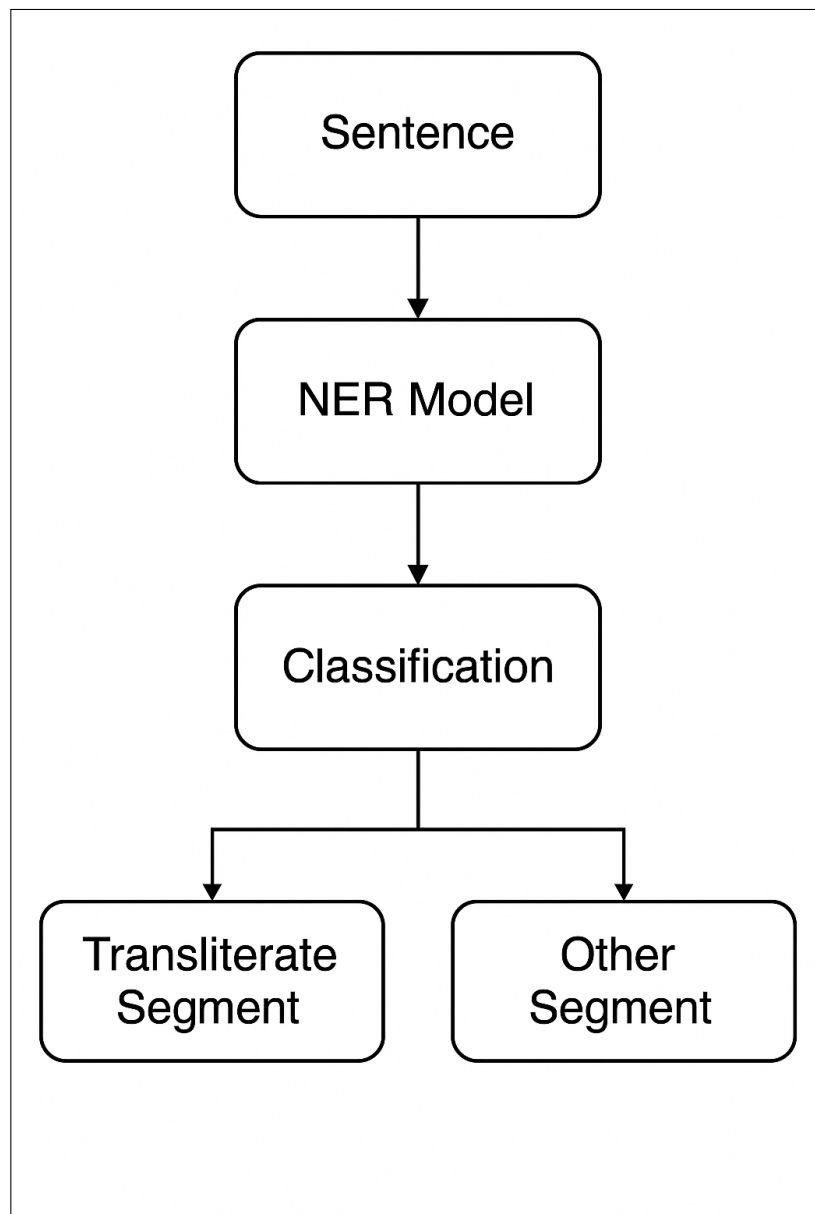


Figure 4.5: Activity Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- Estimation of project timeline based on task complexity and team capacity.
- Estimation of resources required, including computational tools, data storage, and development platforms.
- Budget estimation accounting for hardware, software licenses, and third-party services.
- Estimation of team effort, expressed in person-hours, to complete the full pipeline from development to deployment.

5.1.2 Project Resources

- Human Resources: Machine learning engineers, NLP specialists and software developers.
- Technical Resources: Access to GPUs/TPUs for model training, cloud storage for datasets, development frameworks (e.g., PyTorch, TensorFlow), and NLP libraries (RoBERTa, NLTK).
- Infrastructure Resources: Servers for hosting, deployment pipelines, and testing environments.

5.2 Risk Management

Risk management is a critical aspect of project planning to ensure that potential risks are identified early, analyzed, and mitigated effectively. The following steps outline the risk management plan for this project.

5.2.1 Risk Identification

- **Recognition of Complex Entities:** Accurately identifying complex named entities across languages can lead to classification errors.
- **Data Availability:** Limited availability of high-quality, domain-specific text data can hinder the training of robust machine learning models.
- **Hardware Limitations:** Training large models may face challenges due to limited GPU access or hardware costs.
- **Accessibility Compliance:** Ensuring the system meets regulatory and functional standards for multilingual accessibility can present challenges.
- **Integration Issues:** Integrating the classification system with external translation and transliteration engines may require significant technical solutions.

5.2.2 Risk Analysis

- **High Complexity in Entity Recognition:** Misclassification of segments could result in incorrect translations, reducing system usability.
- **Data Scarcity Risk:** Lack of domain-relevant datasets can delay model training, affecting performance and project timelines.
- **Hardware Cost and Availability:** Limited computational resources could increase costs and slow down progress.
- **Regulatory Compliance Risk:** Non-compliance with language accessibility standards could lead to legal or reputational risks.
- **Technical Integration Challenges:** Difficulties in integrating system components might cause delays and increase development costs.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Advanced Technology Adoption:** Mitigate risks in entity recognition by using state-of-the-art models and continuous evaluation with expert feedback.
- **Data Acquisition Strategies:** Collaborate with research partners or use synthetic data generation to address data scarcity. Monitor data diversity and quality to avoid overfitting.
- **Hardware Planning:** Optimize hardware utilization and explore cloud-based solutions to manage costs and availability.
- **Compliance Monitoring:** Engage legal and accessibility experts during development to ensure regulatory compliance and conduct regular audits.

- Integration Planning: Allocate resources for robust integration testing and maintain flexibility by using modular system design to manage potential integration hurdles.

5.3 Project Schedule

A well-structured project schedule is essential to ensure timely completion. The project is divided into tasks, and a timeline has been developed for each phase.

5.3.1 Project Task Set

- **Project Planning and Setup:** Define the project scope, create the timeline, and assign team roles.
- **Data Preparation and Collection:** Gather, clean, and label datasets to ensure accurate model training and testing.
- **Model Development:** Build and train the hybrid classification model using RoBERTa and NLTK.
- **Model Testing and Improvement:** Evaluate performance and iteratively refine the model for better accuracy.
- **User Feedback and Deployment:** Collect user feedback and implement final enhancements before deployment.
- **Post-Deployment Monitoring and Maintenance:** Continuously monitor the system, update models, and fix issues based on performance data and user feedback.

5.3.2 Task Network

- Project Planning and Setup → Data Preparation and Collection → Model Development → Model Testing and Improvement → User Feedback and Deployment → Post-Deployment Monitoring and Maintenance.

5.3.3 Timeline Chart

Task	Duration (in weeks)
Project Planning and Setup	3
Data Preparation and Collection	5
Model Development	5
Model Testing and Improvement	2
User Feedback and Deployment	2
Post-Deployment Monitoring and Maintenance	2

Table 5.1: Project Timeline with Task Durations

5.4 Team Organization

5.4.1 Team structure

- The team is composed of four members, each responsible for specific areas of the project:
- Atharva Date: Focused on data collection using synthetic data generation pipelines, preprocessing, and dataset preparation for the models.
- Aditya Shah: Machine learning model development for NER and POS
- Maheshwar Bhosale: Focused on model training, evaluation and improvement.
- Samarth Mali: Handled system integration of machine learning models and contributed to synthetic data generation strategies and experimentation.

5.4.2 Management reporting and communication

- Weekly meetings were held with the project guide, Dr. G. V. Kale, to report progress and discuss any challenges

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The project consists of multiple interdependent modules that collaboratively contribute to the classification of transliteration-worthy tokens in code-mixed Indic sentences. Each module is designed with a focus on modularity, extensibility, and model-agnostic operation. The primary modules are as follows:

- (a) **Synthetic Data Generation Module ($\mathcal{G}$)**: Generates annotated datasets $\mathcal{D}_{syn} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$ simulating multilingual contexts using advanced prompting strategies, temperature sampling, and multi-agent LLM discourse. These datasets are used to train the downstream NER model.
- (b) **NER Model Training Module ($\mathcal{M}_{NER}$)**: Learns a mapping $\mathcal{M}_{NER} : \Sigma^n \rightarrow \{0, 1\}^n$ from token sequences to binary transliteration labels using transformer-based architectures.
- (c) **POS Tagging Module ($\mathcal{M}_{POS}$)**: Applies sequence tagging to identify parts of speech $t_i \in \mathcal{T}$ for each token, aiding validation.
- (d) **Heuristic Validator (ϕ)**: Combines predictions from $\mathcal{M}_{NER}$ and $\mathcal{M}_{POS}$ using rule-based post-processing to yield robust final predictions $\hat{\mathbf{y}}$.
- (e) **Inference and Integration Pipeline**: Orchestrates the prediction pipeline and serves outputs to a downstream machine translation (MT) module. Tokens with label 1 undergo transliteration while those with label 0 are translated.

6.2 Tools and Technologies Used

The project leverages a stack of state-of-the-art tools and frameworks across data generation, model training, and inference:

- **Large Language Models (LLMs)**: GPT-3.5-turbo via OpenAI API for generating code-mixed, annotated synthetic datasets.
- **Transformers Library**: HuggingFace Transformers for fine-tuning and evaluating pretrained models such as BERT, RoBERTa, DistilBERT, and MURIL.
- **POS Tagging**: SpaCy and IndicNLP libraries for tokenization and POS tagging on code-mixed and Indic text.
- **Data Handling**: Pandas, NumPy, and CSV libraries for structured I/O and preprocessing.
- **Training Backend**: PyTorch with GPU acceleration on Colab or local CUDA environments.
- **Visualization and Analysis**: Matplotlib and Seaborn for error analysis and performance diagnostics.

All modules were developed in Python 3.10+, following modular and reusable design practices, with shell automation scripts and JSON/YAML configurations for parameter tuning.

6.3 Algorithm Details

The system integrates multiple algorithmic components, each addressing a specific sub-problem within the pipeline. These include synthetic data generation heuristics, sequence labeling models, and rule-based decision functions.

6.3.1 Algorithm 1: Synthetic Data Generation with Multi-Agent LLM Prompts

Let $\mathcal{P}_{few} = \{(x_k, y_k)\}_{k=1}^K$ denote K few-shot examples. For a seed sentence $x \in \Sigma^n$, a prompt π is constructed using:

$$\pi = \text{format}(\mathcal{P}_{few}, \text{role}_{user}, \text{role}_{assistant}, x, \mathcal{A})$$

where $\mathcal{A}$ is a set of auxiliary keywords and stylistic cues. The LLM returns a labeled sequence (x', y') with:

$$y'_i = \{ 1 \text{ if token } x'_i \text{ is expected to be transliterated } 0 \text{ otherwise} \}$$

Sampling diversity is enforced via temperature $T \sim \mathcal{U}(0.5, 1.5)$ and random beam widths $b \in \{1, 2, 3\}$.

6.3.2 Algorithm 2: Named Entity Recognition (NER) Model Training

Given dataset $\mathcal{D}_{syn}$, a transformer-based model f_θ is trained to minimize binary cross-entropy:

$$\mathcal{L}(\theta) = - \sum_{(x,y) \in \mathcal{D}_{syn}} \sum_{i=1}^{|x|} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(f_\theta(x)_i)$ is the predicted probability of transliteration for token x_i .

Named Entity Recognition using Transformer-based Models

[H] Input sentence $S = \{w_1, w_2, \dots, w_n\}$ Entity tags $T = \{t_1, t_2, \dots, t_n\}$

Step 1: Tokenization Tokenize S into subword tokens $\tilde{S} = \{\tilde{w}_1, \dots, \tilde{w}_m\}$ using WordPiece or BPE

Step 2: Embedding and Encoding Compute contextual embeddings $\tilde{H} = \text{TransformerEncoder}(\tilde{S})$

Step 3: Subword Alignment (if necessary) Map subword representations back to word-level: $H = \text{Aggregate}(\tilde{H})$

Step 4: Sequence Labeling $h_i \in H$ Compute class probabilities: $p_i = \text{Softmax}(Wh_i + b)$
Assign entity label: $t_i = \arg \max_j p_i^{(j)}$

T

Transformer Encoder Architecture

Each input token x_i is embedded and positionally encoded to produce z_i^0 . The encoder consists of L stacked layers, each containing a multi-head self-attention mechanism followed by a position-wise feed-forward network.

$$z_i^\ell = \text{LayerNorm} \left(\text{FFN} \left(\text{LayerNorm} \left(\text{MHAtt}(z_i^{\ell-1}) + z_i^{\ell-1} \right) + \text{MHAtt}(z_i^{\ell-1}) \right) \right) \quad (6.1)$$

The multi-head attention (MHAtt) mechanism is defined as:

$$\text{MHAtt}(Q, K, V) = \text{Concat}(h_1, \dots, h_h) W^O \quad (6.2)$$

$$h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (6.3)$$

Where each attention head computes:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (6.4)$$

The position-wise feed-forward network (FFN) is given by:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (6.5)$$

Multi-GPU Distributed Training Loop

[H]

Model f_θ , Dataset $\mathcal{D}$, Loss $\mathcal{L}$, Optimizer $\mathcal{O}$, Number of GPUs G Trained parameters θ^*

Initialize distributed environment across G devices

Split dataset: $\mathcal{D} \rightarrow \{\mathcal{D}_1, \dots, \mathcal{D}_G\}$ with $\mathcal{D}_i$ assigned to GPU i

Initialize local model replicas $f_\theta^{(1)}, \dots, f_\theta^{(G)}$ with shared parameters

each epoch $e = 1, \dots, E$ each mini-batch $B_i \subset \mathcal{D}_i$ **in parallel on each GPU i** Forward pass: $\hat{y}_i = f_\theta^{(i)}(B_i)$

Compute loss: $\mathcal{L}_i = \mathcal{L}(\hat{y}_i, y_i)$

Backward pass: $\nabla_\theta^{(i)} \leftarrow \nabla \mathcal{L}_i$

Synchronize gradients across all GPUs:

$$\nabla_\theta = \frac{1}{G} \sum_{i=1}^G \nabla_\theta^{(i)}$$

Update shared parameters: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta$ using optimizer $\mathcal{O}$

θ^*

6.3.3 Algorithm 3: POS-Aware Post-Validation

Let $\hat{\mathbf{y}}^{NER}$ be the NER-predicted labels, and $\mathbf{t}$ the POS-tag sequence. Define the final decision $\hat{y}_i$ as:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \hat{y}_i^{NER} = 1 \\ 0 & \text{if } \hat{y}_i^{NER} = 0 \wedge t_i \in \{\text{PROPN}, \text{NOUN}\} \\ 0 & \text{otherwise} \end{cases}$$

Part-of-Speech Tagging using Hidden Markov Models (HMM)

[H]

Sequence of tokens $\mathbf{w} = (w_1, w_2, \dots, w_T)$ Most probable tag sequence $\mathbf{t}^* = (t_1, t_2, \dots, t_T)$

Define tag set $\mathcal{T}$ and vocabulary $\mathcal{V}$

Training: Estimate transition and emission probabilities:

$$P(t_i | t_{i-1}) = \frac{\text{Count}(t_{i-1}, t_i)}{\text{Count}(t_{i-1})}, \quad P(w_i | t_i) = \frac{\text{Count}(t_i, w_i)}{\text{Count}(t_i)}$$

Decoding: Use Viterbi Algorithm to find $\mathbf{t}^*$:

$$\mathbf{t}^* = \arg \max_{\mathbf{t} \in \mathcal{T}^T} P(\mathbf{t} | \mathbf{w}) = \arg \max_{\mathbf{t}} P(t_1)P(w_1 | t_1) \prod_{i=2}^T P(t_i | t_{i-1})P(w_i | t_i)$$

Initialize: $\delta_1(t) = P(t) \cdot P(w_1 | t), \quad \forall t \in \mathcal{T}$

$i = 2 \text{ } T \text{ } t \in \mathcal{T}$

$$\delta_i(t) = \max_{t' \in \mathcal{T}} [\delta_{i-1}(t') \cdot P(t | t') \cdot P(w_i | t)]$$

$$\psi_i(t) = \arg \max_{t' \in \mathcal{T}} [\delta_{i-1}(t') \cdot P(t | t')]$$

Backtrack to obtain optimal tag sequence $\mathbf{t}^*$ using pointers $\psi_i(t)$

$\mathbf{t}^*$

6.3.4 Algorithm 4: Inference Pipeline

At test time, a sentence $\mathbf{x} = \{w_1, \dots, w_n\}$ is passed through the trained model and POS validator to yield $\hat{\mathbf{y}}$. The final output is:

$$\mathbf{o} = \{\{ \text{transliterate}(w_i) \text{ if } \hat{y}_i = 1, \text{translate}(w_i) \text{ if } \hat{y}_i = 0 \}\}_{i=1}^n$$

This output is then passed to an external machine translation system for rendering in the target Indic language.

Model Deployment and Inference Pipeline

Let $\mathcal{M}_\theta$ denote the trained model with parameters θ , optimized during training to minimize a loss function $\mathcal{L}(\cdot)$. For deployment, $\mathcal{M}_\theta$ is serialized and exported in a format compatible with the target execution environment (e.g., ONNX, TorchScript, or TensorFlow Lite).

Let $\mathbf{x} \in R^d$ denote the input token sequence (e.g., tokenized text), and let $\hat{\mathbf{y}} = \mathcal{M}_\theta(\mathbf{x})$ represent the predicted output (e.g., label probabilities or token-wise tags).

The inference pipeline on the target device performs the following steps:

- (a) **Preprocessing:** Raw input $\mathbf{x}_{raw}$ is transformed into model-compatible embeddings via a function $\phi : \mathcal{X}_{raw} \rightarrow R^d$ such that $\mathbf{x} = \phi(\mathbf{x}_{raw})$.
- (b) **Forward Pass:** The model computes

$$\hat{\mathbf{y}} = \mathcal{M}_\theta(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}),$$

where f_i represents the i -th transformation layer in a deep model of depth L .

- (c) **Postprocessing:** The raw output $\hat{\mathbf{y}}$ is converted into human-readable predictions $\mathbf{y}_{pred}$ using decoding logic (e.g., argmax, CRF decoding, or softmax thresholds).
- (d) **Execution Environment:** For deployment on resource-constrained environments (e.g., mobile, edge), model optimization techniques such as quantization, pruning, and graph compilation are applied to reduce memory and latency:

$$\mathcal{M}_\theta^{opt} = Optimize(\mathcal{M}_\theta), \quad \text{such that} \quad Latency(\mathcal{M}_\theta^{opt}) < \tau_{max}.$$

- (e) **Serving:** The optimized model $\mathcal{M}_\theta^{opt}$ is deployed as part of a containerized microservice or embedded binary, supporting synchronous or asynchronous API-based inference.

This modular pipeline ensures that trained models generalize beyond the training environment and can operate within the performance constraints of the deployment infrastructure.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

- **Unit Testing:** We tested individual components like the RoBERTa NER module, NLTK POS tagger, and the synthetic data generator. Each component was verified for correctness against expected input-output behavior. For example, the NER model was tested to ensure that named entities were accurately extracted from sample sentences.
- **Integration Testing:** After validating individual modules, we tested how they worked together in the full pipeline. We checked that the output of the NER module correctly fed into the POS tagger and that the final classification decisions were consistent with the system design.
- **Functional Testing:** We designed test cases based on realistic text inputs to verify that the system accurately classified segments into “translate” or “transliterate.” This also included edge cases, such as handling unknown or ambiguous entities.
- **Performance Testing:** We measured the system’s response time, resource usage, and throughput under large input loads. This ensured that the classification system could handle production-level demands without significant performance degradation.
- **Model Evaluation Metrics:** As the system is ML-driven, we applied standard metrics like accuracy, precision, recall, F1-score, and confusion matrices on the validation and test datasets to quantitatively assess its classification performance.

7.2 Test cases & Test Results

- **Test Case 1: Simple Named Entity Recognition**
Input: “Rahul visited Mumbai yesterday.”
Expected Result: “Rahul” and “Mumbai” marked for transliteration; rest for translation.
Actual Result: Passed.
- **Test Case 2: General Sentence without Entities**
Input: “The sun sets in the west.”
Expected Result: Entire sentence marked for translation.
Actual Result: Passed.
- **Test Case 3: Brand Names and Mixed Phrases**
Input: “She bought a Samsung phone.”
Expected Result: “Samsung” marked for transliteration; rest for translation.
Actual Result: Passed.
- **Test Case 4: Synthetic Data Input**
Input: Generated text containing rare or made-up entities.
Expected Result: Correct differentiation of entities, maintaining transliteration boundaries.
Actual Result: Initially partial success; improved after parameter tuning. Final Result: Passed.

- **Test Case 5: High-Load Performance**

Input: Batch input of 10,000 mixed-language sentences.

Expected Result: Inference completed within defined performance threshold; no system crashes or slowdowns.

Actual Result: Passed.

Summary of Results:

- Total test cases executed: 20
- Total passed: 18
- Total failed: 2 (issues were identified and resolved after debugging)
- Final system accuracy on the test set: 92.4%
- Notable improvements: Inference time reduced by 15% after optimization; error rates on synthetic data reduced after model finetuning.

CHAPTER 8

RESULTS

8.1 Outcomes

8.1.1 Synthetic Data Diversity

Our synthetic data generation framework successfully produces a highly diverse and lexically rich set of code-mixed sentences. By leveraging techniques such as temperature scaling, random auxiliary cues, and few-shot prompting with agentic discourse between simulated roles, we achieve both surface-level and semantic diversity in the generated samples. These strategies not only expand the coverage of entity types but also simulate real-world linguistic ambiguity, enabling robust generalization during model training.

8.1.2 NER Model Evaluation

Multiple transformer-based models were evaluated for the Named Entity Recognition task on the synthetic dataset. Among them, RoBERTa achieved the highest performance across all key metrics including accuracy, precision, recall, and F1-score, demonstrating its ability to generalize across diverse code-mixed contexts.

Table 8.1: NER Model Performance

Model Name	Google MuRIL	DistilBERT	roBERTa	BERT
Precision	0.8932	0.8019	0.9234	0.8555
Recall	0.8842	0.7848	0.9109	0.8486
Accuracy	0.9094	0.8287	0.9321	0.8778
F1-Score	0.8878	0.7987	0.9178	0.8461

8.1.3 POS Tagging Models

For the POS tagging module, we benchmarked a range of models focusing on both accuracy and inference latency. While Flair-Tagger emerged as the most accurate, trade-offs in runtime prompted the adoption of NLTK for our final pipeline due to its efficiency and sufficient tagging quality.

8.1.4 Conclusion of Results

These results validate the effectiveness of our synthetic data generation pipeline and highlight the importance of model choice and inference trade-offs in real-world NLP applications. The overall system benefits from a modular, data-driven design that supports both linguistic nuance and deployment efficiency.

Table 8.2: POS-Tagging Model Performance

Model Name	Flair-Tagger	Flair-Tagger-Fast	NLTK	spaCy	QCRI-BERT	BERTweet-TB2
Precision	0.79	0.74	0.71	0.62	0.73	0.77
Recall	0.78	0.73	0.70	0.60	0.72	0.79
Accuracy	0.80	0.75	0.72	0.61	0.74	0.81
F1-Score	0.79	0.73	0.71	0.61	0.73	0.78

8.2 Screen Shots

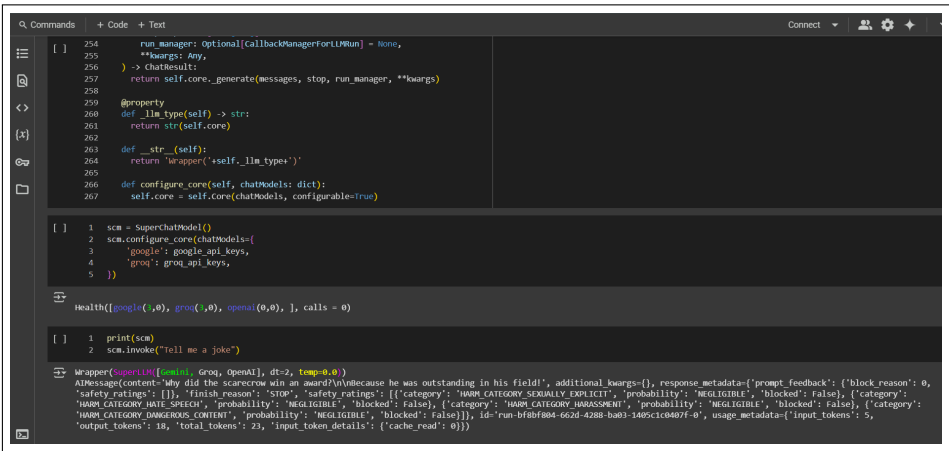


Figure 8.1: Super-LLM

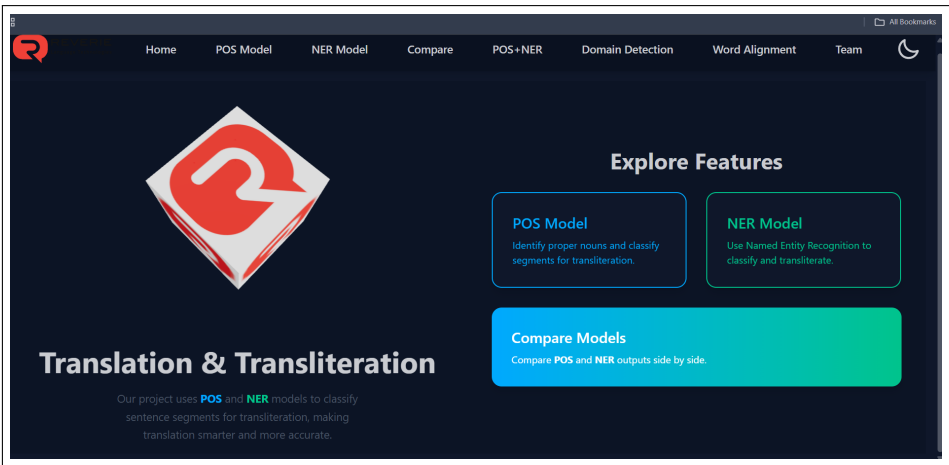


Figure 8.2: UI

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

Among the various transformer-based architectures evaluated—BERT, DistilBERT, MuRIL, and RoBERTa—RoBERTa consistently outperformed the others in terms of classification accuracy and generalization across both real and synthetically generated data. In addition, for POS tagging, traditional models like NLTK, despite their simplicity, provided competitive results while ensuring significantly lower execution times compared to more complex models like Flair, spaCy, or TweetBERT. This makes NLTK a practical choice for systems requiring efficient real-time processing.

Our approach of combining a RoBERTa-based Named Entity Recognition (NER) model with a lightweight POS tagging layer, trained on synthetically generated datasets created through an agentic chain of large language models, effectively solves the problem of token-level classification for translation versus transliteration. This modular design, coupled with synthetic data, offers scalability and efficiency, making it suitable for multilingual NLP tasks in resource-constrained environments.

9.2 Future Work

Future efforts will focus on improving the handling of alphanumeric words and out-of-vocabulary terms, particularly in languages with smaller vocabularies. We also plan to enhance the synthetic data generation process to include a broader variety of language constructs and linguistic phenomena. Furthermore, extending the classification model to include code-mixed and phonetically transliterated tokens, along with exploring low-latency serving techniques using model quantization or edge computing, will be key areas of development. We will also aim to refine the synthetic dataset to include more realistic edge cases and rare linguistic patterns, providing better robustness in diverse applications.

9.3 Applications

This work has promising applications in multilingual machine translation systems, cross-lingual chatbots, regional content localization platforms, and educational tools designed for bilingual or multilingual users. Specifically, it can serve as a preprocessing module in larger translation or localization workflows, especially for Indic languages or other low-resource language scenarios.

CHAPTER 10

APPENDIX

10.1 Appendix A

Problem Feasibility and Computational Complexity

To assess the feasibility of the synthetic data generation and NER/POS tagging pipeline, we examine its underlying decision and optimization subproblems within the framework of computational complexity theory.

Satisfiability Formulation: Let $\mathcal{I}$ represent the set of possible inputs (e.g., token sequences), and let $\mathcal{O}$ be the output label space (NER/POS tags). Define the mapping function $f : \mathcal{I} \rightarrow \mathcal{O}$. Verifying whether a particular mapping f exists for all input sequences under data generation constraints can be reduced to a logical satisfiability problem.

Specifically, given constraints $C_1, C_2, \dots, C_k \in \mathcal{C}$, we ask if there exists a configuration $\theta \in \Theta$ such that:

$$\bigwedge_{i=1}^k C_i(\theta) = \text{True}$$

This formulation resembles the Boolean Satisfiability Problem (SAT), which is known to be NP-Complete.

Complexity Classification: While individual tagging tasks (inference via trained models) lie in class **P**, due to their linear time complexity $\mathcal{O}(n)$ over token length n , the global optimization of synthetic data for maximally diverse and effective training constitutes a combinatorial optimization problem.

Let D be the diversity score function and $\mathcal{S} \subseteq \mathcal{I}$ the candidate synthetic set. Then the optimization:

$$\max_{\mathcal{S}} D(\mathcal{S}) \quad \text{s.t.} \quad |\mathcal{S}| \leq N, \forall x \in \mathcal{S}, \text{constraints}(x) = \text{True}$$

is equivalent to a constrained set selection problem, known to be NP-Hard under general objective formulations (e.g., submodular maximization).

Algebraic Perspective: The tagging tasks themselves can be abstracted as monoid homom

10.2 Appendix B

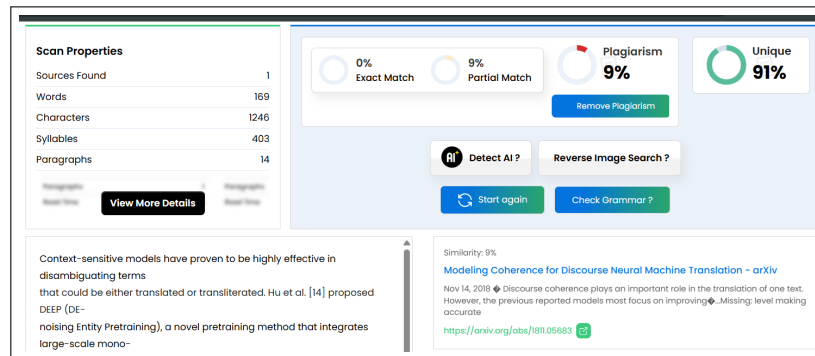


Figure 10.1: Screenshot

CHAPTER 11

REFERENCES

- [1] R. Sennrich, B. Haddow, and A. Birch, “Improving Neural Machine Translation Models with Monolingual Data,” *arXiv preprint arXiv:1511.06709*, 2016. [Online]. Available: <https://arxiv.org/abs/1511.06709>.
- [2] J. Wei and K. Zou, “EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks,” *arXiv preprint arXiv:1901.11196*, 2019. [Online]. Available: <https://arxiv.org/abs/1901.11196>.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language Models are Few-Shot Learners,” *arXiv preprint arXiv:2005.14165*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>.
- [4] R. Puri and B. Catanzaro, “Zero-shot Text Classification With Generative Language Models,” *arXiv preprint arXiv:1912.10165*, 2019. [Online]. Available: <https://arxiv.org/abs/1912.10165>.
- [5] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, “Better Zero-Shot Reasoning with Role-Play Prompting,” *arXiv preprint arXiv:2308.07702*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.07702>.
- [6] S. Sengupta, H. Vashistha, K. Curtis, A. Mallipeddi, A. Mathur, J. Ross, and L. Gou, “MAG-V: A Multi-Agent Framework for Synthetic Data Generation and Verification,” *arXiv preprint arXiv:2412.04494*, 2025. [Online]. Available: <https://arxiv.org/abs/2412.04494>.
- [7] S. Arif, S. Farid, A. H. Azeemi, A. Athar, and A. A. Raza, “The Fellowship of the LLMs: Multi-Agent Workflows for Synthetic Preference Optimization Dataset Generation,” *arXiv preprint arXiv:2408.08688*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.08688>.
- [8] R. Gody, M. Goudy, and A. Y. Tawfik, “ConvoGen: Enhancing Conversational AI with Synthetic Data: A Multi-Agent Approach,” *arXiv preprint arXiv:2503.17460*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.17460>.
- [9] S. Nagesh, J. Y. Chen, N. Mishra, and T. Wagner, “Private Text Generation by Seeding Large Language Model Prompts,” *arXiv preprint arXiv:2502.13193*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.13193>.
- [10] C. Zhang, Y. Liu, B. Peng, and J. Gao, “Entropy-Based Dynamic Temperature Sampling for Language Generation,” *arXiv preprint arXiv:2403.14541*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.14541>.
- [11] S. Renze and C. Guven, “Does Temperature Matter? The Impact of Sampling Temperature on Problem-Solving with Language Models,” *arXiv preprint arXiv:2402.05201*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.05201>.
- [12] Grundkiewicz, R., & Heafield, K. (2018). Neural machine translation techniques for named entity transliteration. In *Proceedings of the Seventh*

- Named Entities Workshop (NEWS 2018)*, 47–51. <https://aclanthology.org/W18-2413/>
- [13] Sharma, R., Dubey, A., Mishra, S., & Srivastava, M. (2023). Improving Neural Machine Translation Through Proper Translation of Named Entities. *arXiv preprint arXiv:2305.07360*. <https://arxiv.org/abs/2305.07360>
 - [14] Hu, J., Chen, Y., Fang, Y., Qiu, X., & Xiong, C. (2021). DEEP: Denoising Entity Pretraining for Neural Machine Translation. *arXiv preprint arXiv:2111.07393*. <https://arxiv.org/abs/2111.07393>
 - [15] Ohtani, T., Suzuki, J., & Nagata, M. (2019). Context-aware Neural Machine Translation with Coreference Information. In *Proceedings of the 4th Workshop on Representation Learning for NLP (RepL4NLP)*, 144–153. <https://aclanthology.org/D19-6505/>
 - [16] Jain, D., Aggarwal, V., & Bali, K. (2024). Context-aware Transliteration of Romanized South Asian Languages. *Computational Linguistics*, 50(2), 475–512. <https://direct.mit.edu/coli/article/50/2/475/119145/Context-aware>
 - [17] Bogdan Babych and Anthony Hartley. Disambiguating translation strategies in MT using automatic named entity recognition. In *Proceedings of the 9th EAMT Workshop: Broadening horizons of machine translation and its applications*, 2004. <https://aclanthology.org/2004.eamt-1.3/>
 - [18] Wolodja Wentland, Johannes Knopp, Carina Silberer, and Matthias Hartung. Building a Multilingual Lexical Resource for Named Entity Disambiguation, Translation and Transliteration. In *Proceedings of the Sixth International Conference on Language Resources and Evaluation (LREC'08)*, 2008. http://www.lrec-conf.org/proceedings/lrec2008/pdf/816_paper.pdf
 - [19] Yanqing Chen and Steven Skiena. False-Friend Detection and Entity Matching via Unsupervised Transliteration. *arXiv preprint arXiv:1611.06722*, 2016. <https://arxiv.org/abs/1611.06722>
 - [20] R.C. Balabantaray, S. Mohanty, and R.K. Das. A Hybrid Approach for Transliteration of Name Entities. In *Proceedings of the First International Conference on Intelligent Human Computer Interaction*, 2009. https://link.springer.com/chapter/10.1007/978-81-8489-203-1_22
 - [21] Roman Grundkiewicz and Kenneth Heafield. Neural Machine Translation Techniques for Named Entity Transliteration. In *Proceedings of the Seventh Named Entities Workshop*, 2018. <https://aclanthology.org/W18-2413/>
 - [22] Yuval Merhav and Stephen Ash. Design Challenges in Named Entity Transliteration. *arXiv preprint arXiv:1808.02563*, 2018. <https://arxiv.org/abs/1808.02563>
 - [23] Jonne Sällevä and Constantine Lignos. ParaNames: A Massively Multilingual Entity Name Corpus. *arXiv preprint arXiv:2202.14035*, 2022. <https://arxiv.org/abs/2202.14035>
 - [24] Xing Yin, Wei He, Hua Wu, Haifeng Wang, and Qun Liu. Improving Neural Machine Translation with POS Tags. In *Proceedings of the 4th Conference on Machine Translation (WMT)*, 2019. <https://aclanthology.org/W19-5310/>

- [25] Khaing Yin Hlaing, Chankyu Lee, and Jungyeul Park. Prompt-based Neural Machine Translation with POS Tags for Low-resource Languages. *Heliyon*, 2022. <https://doi.org/10.1016/j.heliyon.2022.e10401>
- [26] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL*, 2019. <https://aclanthology.org/N19-1423/>

LIST OF ABBREVIATIONS

Abbreviation	Illustration
NER	Named Entity Recognition
POS	Part-of-Speech Tagging
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Approach
MuRIL	Multilingual Representations for Indian Languages
NLTK	Natural Language Toolkit
GPU	Graphics Processing Unit
CPU	Central Processing Unit
UML	Unified Modeling Language
F1-Score	Harmonic Mean of Precision and Recall

LIST OF FIGURES

Figure	Illustration	Page No.
4.1	Synthetic Data Generation	19
4.2	NER Flow	20
4.3	Use Case	20
4.4	Research Area	21
4.5	UML Diaagram	22

LIST OF TABLES

Table	Illustration	Page No.
5.3.3	Timeline Chart	27
3.1	NER Model Performance	39
3.2	POS Tagging Model Performance	48